SALESFORCE ORCHESTRATOR MIGRATION GUIDE

Purpose

This guide documents how to migrate Salesforce reverse ETL orchestration from Airflow-style custom tasks to the Databricks Salesforce orchestrator in this repository.

Scope:
- function-based execution
- operator-based execution
- Unity Catalog connection-based authentication

Migration Summary

Source Pattern (Airflow/Custom)

Typical legacy flow:
1. extract records from warehouse or table
2. call Salesforce client/custom script
3. manually manage batching, retries, and polling
4. surface success/failure in scheduler logs

Common issues:
- repeated custom logic per DAG
- inconsistent error handling and observability
- credential sprawl
- difficult standardization across teams

Target Pattern (Databricks)

Use the package primitives in salesforce_integration:
- Functions:
  - salesforce_upsert
  - salesforce_insert
  - salesforce_update
  - salesforce_delete
- Operators:
  - SalesforceBulkWriteOperator
  - SalesforceUpsertOperator

Concept Mapping

Legacy concept	Databricks equivalent
Airflow PythonOperator for write	python_operator_task with

salesforce_integration.salesforce_* |
| Airflow custom operator | python_operator_task with
salesforce_integration.SalesforceBulkWriteOperator |
Scheduler variable/connection secrets	Unity Catalog connection (conn_id)
Manual polling loop in DAG code	Operator poll() lifecycle
Custom cleanup logic	Operator close() lifecycle

Migration Path A: Function Workflow

Recommended when you want straightforward task-per-operation execution.

Example task shape

```
  python_operator_task:
    main: salesforce_integration.salesforce_upsert
    parameters:
      - name: object_name
        value: "Account"
      - name: external_id_field
        value: "Account_Number_External__c"
      - name: records
        value: '[{"Account_Number_External__c":"A001","Name":"Acme"}]'
      - name: conn_id
        value: "salesforce_m2m_conn"
      - name: wait_for_completion
        value: "true"
```

Use this path when
- each task can run to completion in one call
- minimal orchestration complexity is needed
- you want simplest migration from direct script calls

Migration Path B: Operator Workflow

Recommended when you need lifecycle semantics (open/poll/close) for external orchestration.

Example task shape

```
  python_operator_task:
    main: salesforce_integration.SalesforceBulkWriteOperator
    parameters:
      - name: object_name
        value: "Account"
      - name: operation
        value: "upsert"
```

```
      - name: external_id_field
        value: "Account_Number_External__c"
      - name: records
        value: '[{"Account_Number_External__c":"A001","Name":"Acme"}]'
      - name: conn_id
        value: "salesforce_m2m_conn"
      - name: task_key
        value: "upsert_with_operator"
```

Use this path when
- you need explicit long-running external lifecycle management
- you want standardized polling and cleanup behavior
- you are replacing custom Airflow operator-style code

Authentication Migration

Move from raw credentials to Unity Catalog connection references:

- Old: username/password/token in scheduler variables or env vars
- New: conn_id to UC connection configured in workspace

Result:
- centralized governance
- no credentials in pipeline code
- easier rotation and control

Data Contract Expectations

Records parameter
- pass JSON string (Databricks task parameter)
- function/operator coerces to list of dicts

Field requirements
- upsert requires external_id_field
- update/delete require Salesforce Id field in each record

CSV formatting
- orchestrator emits Salesforce-compatible LF line endings

Validation Checklist

- [ ] UC connection configured and accessible
- [ ] conn_id points to M2M-ready Salesforce connection

- [ ] function job runs success (salesforce_full_workflow)
- [ ] operator job runs success (salesforce_upsert_operator)
- [ ] target records visible in Salesforce object
- [ ] failure behavior tested with invalid payload case

Deployment Commands

```
cd /Users/pravin.varma/Documents/Demo/salesforce_operator_hackathon
python3 setup.py bdist_wheel
databricks bundle validate --profile dogfood
databricks bundle deploy --profile dogfood
```

Run function path:

```
databricks bundle run salesforce_full_workflow --profile dogfood
```

Run operator path:

```
databricks bundle run salesforce_upsert_operator --profile dogfood
```

Notes

- This repository intentionally excludes Salesforce sensor implementation.
- Migration scope is function/operator only.